

Assignment 2-Case Study
Stage 4 Lab Activities
Implementing the SmartCare Domain Layer

DESIGN FIRST -> AI PAIR PROGRAMMING -> REVIEW -> VERIFY | 1hour

A - Revisit Approved UML

Confirm responsibilities, attributes and relationships before coding.

Class	Attributes	Responsibilities
Patient	Patient ID, name, contact details	Store patient information and validate updates.
Practitioner	Practitioner ID, name, specialty	Store practitioner information and validates updates
Appointment	Appointment ID, patient, practitioner, date/time, status	Store booking details and control rescheduling, cancellation and completion*.

Relationships

- Each appointments link to exactly one patient and to one practitioner.
- Patients can have many or zero appointments
- Practitioner can as well have many or zero appointments
- Associations not inheritance

B - Implement Patient: AI OFF

Implement Patient with type hints and basic validation.

The class is within the python file called Classes

C - Implement Practitioner: AI OFF

Implement Practitioner with identifier, name and specialty; no database logic.

The class is within the python file called Classes

D - Implement Appointment: AI ON

Give AI the approved Appointment UML, business rules and explicit constraints. Ask it to implement only Appointment and agreed enum/exception.

E - Review Generated Code

Check model consistency, unsupported features, public state mutation, unnecessary inheritance, invented dependencies and error handling.

All These are within the log

F - Manual Behaviour Checks

Create valid objects, test invalid input, cancel a scheduled appointment and attempt an illegal repeated transition.

Testing Final Code of AlgenCode6

I created a valid Patient, Practitioner and Appointment. The appointment was created with a SCHEDULED status. I tested invalid patient and practitioner objects; each raised a TypeError. I then cancelled a scheduled appointment and confirmed its status changed to CANCELLED. Trying to cancel it again raised InvalidStatusTransitionError. I also checked that a completed appointment cannot be rescheduled. These results matched the expected behaviour.

Things I tested

Test	What to do	Expected result	Actual result	Pass/Fail
Create valid objects	Create a Patient, Practitioner and Appointment with valid details.	Objects are created. Appointment status is SCHEDULED.	Objects are created	P
Blank required input	Enter a blank patient name or appointment ID.	Input is rejected. The terminal asks you to retry.	Input is rejected	P
Invalid date input	Enter "abc" at the date/time prompt.	Displays a message and asks you to retry.	Displays error	P
Cancel an appointment	Call cancel() on a scheduled appointment.	Status becomes CANCELLED. The object still exists.	CANCELLED	P
Repeat cancellation	Call cancel() again on that appointment.	Raises InvalidStatusTransitionError. Status stays CANCELLED.	stays CANCELLED	P
Complete an appointment	Call complete() on a new scheduled appointment.	Status becomes COMPLETED.	COMPLETED.	P
Change a completed appointment	Call cancel() on the completed appointment.	Raises InvalidStatusTransitionError. Status stays COMPLETED.	stays COMPLETED	P
Reschedule appointment	Call reschedule() with a valid new datetime.	The appointment time changes. Status stays SCHEDULED.	The appointment time changes. Status stays SCHEDULED.	P
Reschedule a cancelled appointment	Call reschedule() after a cancelled cancellation.	Raises InvalidStatusTransitionError. Time stays unchanged.	Error	

Reschedule Call reschedule() after a completed completion. appointment

Raises InvalidStatusTransitionError. Time stays unchanged. Error

G - Refactor

Remove unnecessary code and make implementation simpler and design-consistent.

All These are within the log

H - AI Engineering Log

Record prompt, generated contribution, decisions and verification evidence.

AI Error	Prompt to fix	Decision to keep new code	Verification
Appointment attributes are public, so outside code can change its status and other state directly.	"Make Appointment attributes internal using _ names. Keep simple getter methods and update its methods to use the internal attributes." The Code is AigenCodeDraft2	Yes but need to fix and edit other mistakes	Check that getters return the correct values and that cancel() updates _status.
The constructor does not verify that patient and practitioner are the	Check that patient is a Patient and practitioner is a Practitioner. Raise TypeError for invalid objects. Keep the basic coding style	Yes It helps with validity of inputs	Pass a string instead of a Patient or Practitioner. Confirm that each raises TypeError.

correct objects.	Code in AigenCodeDraft3		
Cancelled or completed appointments can still be rescheduled.	Only allow rescheduling when the status is SCHEDULED. Otherwise, raise InvalidStatusTransitionError. Use simple if and elif statements. AigenCodeDraft4	Proposed — keep after testing. This enforces our rule that only scheduled appointments can move.	Reschedule a scheduled appointment successfully. Then try rescheduling cancelled and completed appointments; both should be rejected.
send_reminder() claims a reminder was sent and adds an unsupported notification feature.	“Remove send_reminder() from Appointment. Do not add notification features or dependencies.” AigenCodeDraft5	Proposed — keep the removal. Notifications are excluded by the lab instructions.	
A scheduled appointment can be changed to SCHEDULED again.	Reject SCHEDULED to SCHEDULED using InvalidStatusTransitionError. Keep the existing checks for cancelled and completed appointments. AigenCodeDraft6	Proposed — keep after testing. This enforces our rule against repeated status transitions	All change_status(AppointmentStatus.SCHEDULED) on a scheduled appointment. Confirm that it raises the exception.

 AigenCodeDraft(3).py
Code

Make Appointment attributes internal using `_` names. Keep simple getter methods and update its methods to use the internal attributes.

Worked for 29s >

Updated  AigenCodeDraft(3).py. `Appointment` now stores its five attributes with `_` names, and its getters and methods use those names. The file passes Python's syntax check.

 AigenCodeDraft(3).py
Code

Check that patient is a Patient and practitioner is a Practitioner. Raise `TypeError` for invalid objects. Keep the basic coding style

Worked for 22s >

Updated  AigenCodeDraft(3).py. `Appointment` now checks that `patient` is a `Patient` and `practitioner` is a `Practitioner`, raising `TypeError` for either invalid object. I verified both checks and a valid appointment.

 AigenCodeDraft(3).py
Open file 

Only allow rescheduling when the status is `SCHEDULED`. Otherwise, raise `InvalidStatusTransitionError`. Use simple `if` and `elif` statements.

Worked for 23s >

Updated  AigenCodeDraft(3).py. `reschedule()` now uses simple `if/elif` checks and raises `InvalidStatusTransitionError` unless the appointment is `SCHEDULED`. I verified scheduled, cancelled, and completed cases.

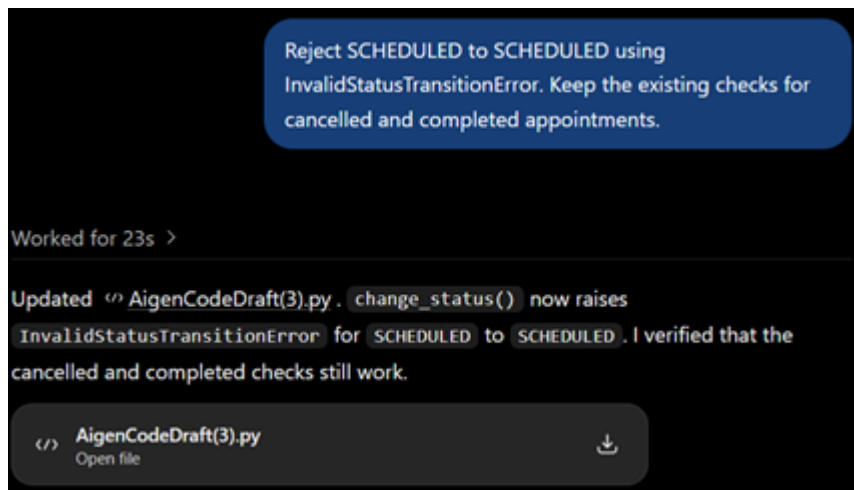
 AigenCodeDraft(3).py
Code

Remove `send_reminder()` from `Appointment`. Do not add notification features or dependencies."

Worked for 22s >

Removed `send_reminder()` from `Appointment` in  AigenCodeDraft(3).py. No reminder or notification references remain, and the file passes Python's syntax check.

 AigenCodeDraft(3).py
Code



Suggested AI prompt

Act as a Python pair programmer. Implement only the Appointment class from the approved SmartCare UML. Use type hints and an AppointmentStatus enum. Cancelled appointments remain as objects. Do not add database, UI, notification or service classes. Protect status transitions and explain any decision not directly visible in the UML.

Reflection

Which AI-generated part did you modify or reject? Why? How did the approved design constrain the AI?

After comparing the AI-generated Appointment code with the authorised UML and business rules, I made modifications. In order to prevent data from being altered directly from outside the class, I used `_` to rename the Appointment properties to internal names. To ensure that the values could still be read, I maintained straightforward getter methods.

In order to verify that the practitioner is a Practitioner object and the patient is a Patient object, I additionally provided validation. The constructor reports a `TypeError` if the incorrect type is supplied.

An `InvalidStatusTransitionError` is raised, and the appointment must not alter. Additionally, I was also changed from being changed from `SCHED`.

The approved UML and the lab criteria did not include notifications, thus I eliminated the `send_reminder()` function. The AI was restricted by the authorised design to the specified

classes, attributes, methods, relationships, and appointment status criteria. This assisted in keeping the AI from incorporating unnecessary features.